

Mission Control Internal CRM — Evidence-Backed Build Status

Audit date: 20 July 2026
System audited: ClinicGrower / The Growth Group Internal CRM (“Mission Control”) only
Repository: <https://github.com/hailolaa/Internal-CRM>
Commit audited: f11771b on main
Assessment basis: current frontend, backend, database schema, migrations, tests, configuration, deployment documentation and Git history

1. Scope and confidence

This report deliberately covers **only this Internal CRM / Mission Control repository**. It does not assess the Clinic Operating System as a separate product. Clinic-facing data is mentioned only where the boss’s Mission Control requirements ask whether client performance data can be pulled into the internal CRM.

The code-level evidence is strong: the repository contains both frontend and backend, a fresh-database schema, migrations, environment templates and many test programs. Evidence about actual production use is weaker because no deployed session, database, secrets manager, provider consoles, live Trello board or named test account was available during the audit.

Status labels used below:

- **Built / usable v1:** frontend, backend and persistence exist for the core workflow.
- **Partial:** meaningful working code exists, but the requested business outcome is incomplete or not proven live.
- **Foundation only:** schema, configuration, isolated fields or reusable components exist, but no complete user workflow.
- **Not built / not evidenced:** no implementation matching the requested capability was found in this repository.

2. Honest headline assessment

Production-ready completion

Mission Control is approximately 52% complete against the full scope in the boss’s message.

This is not a percentage of screens. It is a weighted production-readiness assessment:

Capability group	Weight	Current readiness	Weighted contribution
Leads, contacts and sales pipeline	20%	78%	15.6%
Clients, services and delivery work	20%	70%	14.0%
Communications, calls and follow-up	15%	58%	8.7%
Finance, contracts and commercial documents	15%	25%	3.8%
Client results, integrations and cross-client reporting	15%	35%	5.3%
Team, security, administration and operations	15%	32% production-proven / 65% code foundation	4.8%
Total	100%		52.2%

The system is considerably closer to an **internal alpha** than to the complete management platform described in the message. Core CRM and delivery workflows are real. The largest omissions are finance/profitability, proposals/SOW/contracts, external task-management integration, Clinic OS aggregation, support, Fireflies and production operations evidence.

Remaining effort

Work type	Planning range
Remaining development for the full requested Mission Control scope	420–650 hours with Codex assisting
Testing, data validation, bug fixing and production hardening	150–230 hours with Codex assisting
Third-party setup/approval elapsed time	Not included; potentially 2–8 weeks depending on providers

These are **Codex-assisted estimates**, not traditional developer-only estimates. They assume Codex is used continuously to inspect the codebase, implement focused changes, generate migrations and tests, trace integration contracts, fix type/build issues and keep documentation current. A human developer still owns architecture and product decisions, credentials, provider consoles, production deployment, browser acceptance, security review and approval of AI-generated changes. The ranges assume the existing code is retained rather than rewritten wholesale and that requirements are made concrete. Accounting, Fireflies, ClickUp and Clinic OS ingestion estimates will change materially once vendors and data contracts are confirmed. Codex reduces hands-on implementation time; it does not reduce third-party approval or stakeholder decision time.

What can launch now

After a successful staging and security release gate, Mission Control could be launched internally as an alpha for:

- lead and contact entry/import;
- prospect prioritisation and next-best-action guidance;
- sales pipeline/Kanban and deal movement;
- client account creation and account contacts;
- package, service, health, churn and renewal tracking;
- delivery work and internal task workspaces;
- task comments, mentions, attachments and activity history;
- inbound email, call records and WhatsApp in controlled/provider-configured modes;
- Google Drive client folders;
- team, roles, permissions and audit history.

It is **not ready to be represented as a complete finance, client-results, executive-intelligence or cross-system oversight platform**.

3. Detailed module status

3.1 Internal dashboard

Status: Built / usable v1, not fully production-proven.

- **Purpose:** give leadership a daily view of sales, clients, active work, overdue tasks, audits and next actions.
- **Frontend:** /app is explicitly titled “Mission Control.” It calculates API-backed KPIs for new prospects, won/lost deals, open clients, active projects and overdue tasks; it also presents pipeline stages, upcoming deadlines and next-best actions.
- **Backend/data:** it consumes contacts, pipeline stages/deals, client accounts/services and internal tasks through typed API clients. Next-best actions are deterministic and explainable rather than opaque AI.
- **Dummy/test data:** the page itself is API-backed, but a fresh `db.sql` installs extensive example records. Any demo environment must distinguish seed data from live information.
- **Limitations:** no accounting-grade cash/MRR/profit view, no daily briefing delivery and no Clinic OS fleet metrics. KPI definitions have not been signed off against live company data.
- **Tests:** dashboard card behavior has frontend unit tests; no completed browser/staging evidence was available.
- **Production work:** live-data validation, role-specific views, error/empty-state browser QA, KPI sign-off and monitoring.
- **Suggested owner:** Michael 2/frontend plus lead developer for API/data acceptance.
- **Codex-assisted estimate:** 24–40 development/QA hours.

- **Evidence:** `frontend/app/app/page.tsx`, `frontend/lib/dashboard-cards.ts`, `frontend/lib/dashboard-cards.test.ts`, `frontend/lib/next-best-action.ts`.

3.2 Enquiry and lead management

Status: Built / strong v1.

- Contact/lead CRUD, pagination, search, filters, import batches, duplicate candidates, attribution/consent fields, audit status, follow-up fields, priority scoring and next-best-action logic exist.
- `/app/leads` provides the prospect list; contact create/edit/detail/import/duplicate-review pages are present.
- Backend stores normalized contacts and creates activity/audit records. Website-lead and public-form intake can create leads and retain raw payload context.
- Known limitations are the lack of a complete merge workflow, incomplete universal identity resolution across email/phone/provider IDs, and the need to enforce qualification/lost-reason discipline.
- **Codex-assisted estimate to production-ready:** 24–40 hours.
- **Evidence:** `frontend/app/app/leads/page.tsx`, `frontend/app/app/crm/contacts/**`, `backend/src/modules/contacts/**`, `backend/src/modules/website-leads/**`, tables `contact`, `contact_duplicate_candidate`, `contact_import_batch`, `activity`.

3.3 Sales pipeline and Kanban

Status: Built / usable v1.

- Pipeline and stage APIs support listing/configuration; deals support create/update/move and movement history.
- The frontend `/app/crm/pipeline` is a substantial API-backed Kanban/workspace with filters, ownership, deal value and proposal-oriented views.
- Database models include `pipeline`, `pipeline_stage`, `deal` and `pipeline_deal_movement`.
- Limitations: sales forecasting is basic, proposal status is inferred from deal/stage text in places, and end-to-end won/lost conversion governance needs acceptance testing.
- **Codex-assisted estimate:** 20–36 hours.
- **Evidence:** `frontend/app/app/crm/pipeline/page.tsx`, `backend/src/modules/pipeline/**`, pipeline tests under `backend/src/test/`.

3.4 Contact records

Status: Built / strong v1.

- Full contact profile, notes, tags, account name, lead status, package interest, attribution, consent, follow-up, audit/demo records and linked activities are stored.
- Detail UI includes activity and links into calls, meetings/consults, client accounts and proposals/pipeline.
- Limitations: merge/resolution is weaker than duplicate detection; no formal retention/archive policy is proven.
- **Codex-assisted estimate:** 16–24 hours.
- **Evidence:** contact frontend pages/API types, `backend/src/modules/contacts/**`, `contact`, `activity`, `contact_duplicate_candidate`.

3.5 Company and clinic/client account records

Status: Built / usable v1 with naming debt.

- Mission Control uses the legacy `clinic` table as the account/workspace root and `client_account_profile` for account manager, package, onboarding, health, churn, renewal, contract and notes.
- Users can create a client directly or convert/link a contact; list/detail/package/Drive screens and linked records exist.
- This is real DB/API-backed functionality with audit events and explicit `client_accounts:read/write` permissions.
- Technical limitation: `clinic_id` and clinic-facing terminology remain throughout, which can confuse internal accounts with external client tenants and increases takeover/security risk.
- **Codex-assisted estimate:** 28–48 hours for production polish; a future naming migration is a separate project.

- **Evidence:** docs/mission-control-data-model.md , client-account frontend pages, backend/src/modules/client-accounts/**, tables clinic, client_account_profile, clinic_membership.

3.6 Call and meeting records

Status: Calls built/partial; meetings partial.

- Staff can create, list, view, update and export calls; Twilio-compatible webhooks can ingest call and recording events.
- Call analytics, recordings, transcription, OpenAI intelligence, missed-call follow-up and recording-deletion workflows have routes and persistence.
- Contact records can store sales call/demo entries and consult/appointment records, but there is no general meeting platform integration or canonical meeting object for all client/sales meetings.
- Live Twilio/OpenAI behavior depends on secrets and was not tested here.
- **Codex-assisted estimate:** 40–64 hours, excluding a selected calendar/meeting vendor.
- **Evidence:** frontend/app/app/comms/calls/**, backend/src/modules/calls/**, backend/src/services/openai-call-*.service.ts, tables call, call_recording_deletion_request, appointment, manual_consult_entry.

3.7 Email activity

Status: Partial but materially built.

- Unified inbox/conversation UI and APIs exist. Staff can read, star, archive and reply.
- Brevo outbound service and a signed/secret inbound webhook mapping messages to a workspace exist; inbound email metadata was added in the latest code.
- Background sequences can execute due email/SMS steps.
- Not proven: mailbox-wide synchronization, attachment handling, bounce/delivery reconciliation, provider production setup and full thread matching.
- **Codex-assisted estimate:** 32–56 hours.
- **Evidence:** frontend/app/app/comms/inbox/page.tsx, backend/src/modules/comms/**, backend/src/services/email.service.ts, backend/scripts/setup-brevo-inbound-webhook.mjs, email, sms.

3.8 WhatsApp activity

Status: Partial / strong foundation.

- Conversation/message/reply/settings models exist. Staff can view conversations, send manually, generate AI drafts, approve drafts and retry failed sends.
- Providers support safe log mode, Meta Cloud API and Twilio configuration. Meta webhook signature verification and phone-number-to-workspace mapping exist.
- Production readiness depends on a real approved number, templates, opt-in/opt-out acceptance, webhooks, monitoring and a live test. Default log mode does not send externally.
- **Codex-assisted estimate:** 28–48 hours plus provider approval.
- **Evidence:** backend/src/modules/comms/twilio-whatsapp.provider.ts, whatsapp-ai.service.ts, comms/webhook routes, WhatsApp schema tables and tests.

3.9 Sales tasks and follow-ups

Status: Built / strong v1.

- Internal task list/new/detail pages are API-backed.
- Tasks support client/service/contact linkage, assignee, priority, due date, status, recurrence, QA, missed/escalation flags and archive.
- The task workspace includes comments, mentions, attachments, downloads and activity. Recurring task generation is scheduled daily.
- Gaps: no full notification center/email mention delivery, workload/capacity model or production storage/backup proof for attachments.

- **Codex-assisted estimate:** 20–36 hours.
- **Evidence:** `frontend/app/app/crm/tasks/**`, `frontend/app/app/ops/delivery/page.tsx`, `backend/src/modules/tasks/**`, `docs/task-workspace/design.md`, task-related schema/migration.

3.10 Proposal and statement-of-work creation

Status: Foundation only.

- A normalized `proposal` table exists with contact, deal, client account, package, amount, status and URL fields. Package records store proposal wording. Pipeline and command-palette views can identify proposal-like deals.
- No dedicated proposal/SOW API module or working proposal document builder/send/version/PDF/e-sign workflow was found. A contact link to `/app/proposals` exists even though no such page exists; other links route back to filtered pipeline views.
- **Production work:** requirements, templates, line items, version history, internal approval, branded rendering, send/view tracking and e-sign integration.
- **Codex-assisted estimate:** 56–96 hours.
- **Evidence:** `backend/db.sql` proposal table, `backend/src/modules/packages/**`, pipeline/command-palette proposal filtering, absence of proposal routes in `backend/src/app.ts`.

3.11 Contract management

Status: Partial fields, not a contract-management workflow.

- Client profiles/services store contract status and renewal dates; the UI can update these values.
- There is no contract document/version/signature/counterparty/notice workflow or e-sign provider integration.
- **Codex-assisted estimate:** 48–80 hours.
- **Evidence:** `client_account_profile`, `client_account_service`, package/client-account pages and service validators.

3.12 Invoice and payment tracking

Status: Foundation only for the requested internal use.

- Stripe, payment, deposit, subscription and invoice-number fields are inherited and wired in parts.
- No Mission Control accounts-receivable ledger, invoice builder, VAT/tax treatment, accounting sync, payment allocation or reconciliation workflow is evidenced.
- **Codex-assisted estimate:** 56–96 hours, highly dependent on Xero/QuickBooks/Stripe decisions.
- **Evidence:** billing/deposit modules, Stripe utility/config, `payment`, `deposit_record`, subscription fields.

3.13 Client onboarding

Status: Partial.

- Client records track onboarding status; new-client and package screens collect initial commercial/account data.
- Delivery tasks and SOPs can represent onboarding work.
- Missing is a first-class checklist/template workflow with dependencies, owners, access collection, approvals, due-date automation and completion reporting.
- **Codex-assisted estimate:** 32–56 hours.
- **Evidence:** client-account pages/service, `onboarding_status`, internal task/SOP modules.

3.14 Client account management

Status: Built / usable v1.

- Account profile, manager, services, status, health, churn, renewal, contract, key notes, linked contacts/records, Drive and current/recommended package are implemented.
- All-client summaries include delivery pressure and action-plan context.
- Missing: canonical all-channel timeline, complete client-results feed and formal renewal/support workflows.

- **Codex-assisted estimate:** 28–44 hours.
- **Evidence:** `backend/src/modules/client-accounts/**`, account frontend pages and client account tests.

3.15 Service and package tracking

Status: Built / usable v1.

- Package catalogue supports included features, internal notes, proposal wording, ordering and default/status.
- Client services support owner, dates, recurring value, currency, contract status, service type, archive and filters.
- Missing: scope versioning, entitlements, margin/cost linkage and automated renewal changes.
- **Codex-assisted estimate:** 20–36 hours.
- **Evidence:** `backend/src/modules/packages/**`, client-account services routes/pages, `client_account_service`, package tables.

3.16 Campaign and project delivery

Status: Partial / usable task-based delivery.

- Campaign CRUD exists. Delivery work is managed through client/service-linked internal tasks with board/category, workflow month, QA, missed/escalation and proof/attachment support.
- It is not a full project system: dependencies, milestones, estimates/actual time, resource capacity, budget and portfolio forecasting are missing.
- **Codex-assisted estimate:** 40–64 hours for the intended lightweight delivery MVP.
- **Evidence:** campaign module/pages, `/app/ops/delivery`, task service/schema/tests.

3.17 ClickUp or task-management integration

Status: Not built.

- Mission Control has its own task system, but no ClickUp API, OAuth, webhook or synchronization code was found.
- A product decision is required: Mission Control or ClickUp must be the source of truth before integration work begins.
- **Codex-assisted estimate:** 48–80 hours after decision.

3.18 Team and freelancer workload

Status: Partial.

- Team users, invitations, roles and task assignments exist. Delivery views can be filtered and task QA stores a `freelancer_team_score`.
- There is no freelancer entity, contract/rate, availability, capacity, time tracking or forecast workload model.
- **Codex-assisted estimate:** 48–76 hours.
- **Evidence:** team module/pages, task assignment and QA fields, `freelancer_team_score` tests.

3.19 Staff performance

Status: Foundation only.

- SLA response and call metrics, task completion/overdue/missed flags and QA score fields provide inputs.
- No coherent internal staff scorecard, agreed weighting, trends, goals or fairness controls exist.
- **Codex-assisted estimate:** 32–56 hours after metric definitions.
- **Evidence:** SLA/call metrics, internal task fields and team module.

3.20 Client communication history

Status: Partial but useful.

- Contact activity can group calls, forms, bookings and messages. Inbox covers email/SMS; WhatsApp conversations and sales call/demo records exist.

- No single canonical client-account timeline combines every contact, meeting, delivery note, external email and platform event.
- **Codex-assisted estimate:** 28–48 hours.
- **Evidence:** contact grouped activity service/tests, comms modules, client linked-record endpoints.

3.21 Client support requests

Status: Not built as a dedicated workflow.

- A task can manually represent a support request, but there is no ticket intake, requester/thread, severity, SLA, escalation, portal or support reporting model.
- **Codex-assisted estimate:** 44–72 hours.

3.22 Client health scoring and churn-risk warnings

Status: Partial.

- Client profile stores health and churn risk; dashboard/account views surface them and deterministic next-best-action rules recommend review work.
- Current values are largely entered/maintained fields rather than a validated score calculated from performance, communication, delivery, payment and sentiment data.
- **Codex-assisted estimate:** 28–48 hours plus business calibration.
- **Evidence:** `client_account_profile`, client pages, `frontend/lib/next-best-action.ts`.

3.23 Renewal and notice-period tracking

Status: Renewal partial; notice periods not built.

- Renewal dates and contract statuses are filterable; dashboard deadlines and next actions surface upcoming renewals.
- No notice-period fields, reminder schedule, renewal opportunity pipeline or approval workflow exists.
- **Codex-assisted estimate:** 24–40 hours.

3.24 Finance dashboard

Status: Partial/inherited reporting, not a Mission Control finance dashboard.

- Revenue/financial/report pages and manual spend/deposit/payment data exist in the broader codebase, but they are not in the current Mission Control navigation.
- The internal dashboard reads deal values and service recurring values, not a financial ledger.
- **Codex-assisted estimate:** 56–96 hours after finance-source decisions.

3.25 Revenue, recurring revenue and profitability

Status: Revenue/MRR inputs partial; profitability not built.

- Deal value, client-service recurring value/currency and report infrastructure exist.
- No MRR movement schedule, recognized revenue, direct cost allocation, supplier cost or gross-margin model exists.
- **Codex-assisted estimate:** 56–96 hours after agreeing definitions and accounting system.

3.26 Supplier and freelancer costs

Status: Not built.

- No supplier/freelancer entity, bill, rate card, purchase/cost allocation or approval flow was found.
- **Codex-assisted estimate:** 44–72 hours, or lower if read-only from an accounting platform.

3.27 Client results and marketing performance

Status: Partial.

- The repository has campaign, manual spend/metrics, attribution, calls, consults, revenue and reporting modules inherited as usable building blocks.
- They are scoped to the active `clinic_id`; no proven mapping connects each external Clinic Operating System tenant to the corresponding internal client account and central result view.
- **Codex-assisted estimate:** 56–96 hours after the cross-system data contract.

3.28 Website, SEO, Google Ads, Meta and GBP reporting

Status: Google integrations materially built but unverified live; Meta metric sync incomplete.

- Integration definitions, OAuth start/callback, encrypted tokens, account/property/location selection, sync status and raw payload/error recording exist.
- Current vendor metric-fetch code supports Google Ads, GA4, Google Business Profile and Search Console/SEO. It persists normalized manual metric rows and spend where applicable.
- Meta OAuth/account listing exists, but vendor metric sync explicitly returns “not implemented yet; manual fallback remains available.”
- No staging provider run was available, so “implemented in code” must not be presented as “connected in production.”
- **Codex-assisted estimate:** 48–88 hours for live credentials, validation, scheduled synchronization, reconciliation and Meta completion.
- **Evidence:** `backend/src/modules/integrations/integrations.service.ts`, `integration routes/tests`, `integration fields/raw payload` and `manual metric tables`.

3.29 Account access and credential management

Status: Partial for OAuth/API keys; not a credential vault.

- Mission Control manages internal API keys and encrypted OAuth access/refresh tokens. Google Drive access can be validated and selected.
- Connector tokens are encrypted with AES-GCM using a key derived from the JWT secret. This is better than plaintext but couples encryption and JWT rotation and is not equivalent to a dedicated KMS/vault.
- There is no safe client-password vault workflow. Do not add general credentials to application tables; integrate 1Password/Bitwarden or a managed secrets platform.
- **Codex-assisted estimate:** 28–48 hours for a vault integration and access audit.

3.30 Reporting across all clients

Status: Partial for account operations; not complete for performance.

- All-client account summaries exist and include package/status/health/churn/renewal/delivery/action-plan pressure.
- Cross-client marketing/revenue performance needs a normalized client-results feed and a stable client/tenant identity map.
- **Codex-assisted estimate:** 48–76 hours after ingestion exists.

3.31 Ability to pull data from each Clinic Operating System

Status: Not built as a cross-system integration.

- Generic API keys, public intake routes, webhooks, integration raw payloads and manual metric ingestion are foundations.
- No fleet registry, Clinic OS endpoint credentials, versioned sync contract, tenant-to-client mapping, incremental checkpoint, replay/dead-letter queue or cross-system reconciliation job was found.
- This is the single biggest functional gap between current Mission Control and the boss’s central-oversight vision.
- **Codex-assisted estimate:** 96–168 hours for an initial reliable data feed, excluding changes required in Clinic OS.

3.32 Data warehouse or central data structure

Status: Operational MySQL schema exists; analytics warehouse does not.

- The database has more than 90 operational tables and broad tenant scoping.

- It is a transactional schema inherited from clinic CRM concepts, not a designed central warehouse with client dimensions, source identities, metric facts, snapshots, data quality and lineage.
- **Codex-assisted estimate:** 72–120 hours for a minimum central reporting model and ingestion controls.

3.33 AI summaries and recommendations

Status: Partial and real in code.

- OpenAI-backed routes exist for Growth Brief, show-rate, sales assistant, campaign analyst, LTV optimiser, competitor insights, call intelligence/transcription and WhatsApp replies.
- Leakage insights and next-best actions also have deterministic/fallback behavior and stored provenance in parts.
- Production provider configuration, evaluation datasets, cost limits, prompt/version governance and human approval are not proven.
- **Codex-assisted estimate:** 40–64 hours for production governance and acceptance.
- **Evidence:** AI workspace routes/service, OpenAI services, insights and WhatsApp AI tests.

3.34 Daily executive briefing

Status: Data ingredients partial; briefing not built.

- A daily 07:00 background job computes SLA/no-show/consult/revenue counters, but it does not produce and deliver the requested executive briefing.
- **Codex-assisted estimate:** 24–40 hours once content and delivery channel are approved.

3.35 Alerts, warnings and blocked-task reporting

Status: Partial.

- Dashboard shows overdue tasks and client risks. Internal tasks have missed/escalation/QA states. Insights and alerts pages exist elsewhere in the codebase.
- No single alert inbox with acknowledgement, ownership, escalation, email/Slack delivery and alert lifecycle is evidenced.
- **Codex-assisted estimate:** 28–48 hours.

3.36 Voice command or ChatGPT readiness

Status: API-ready foundations; user capability not built.

- REST APIs, permissions, command-palette search/actions and structured AI services could be exposed through a controlled tool layer.
- No voice capture, ChatGPT connector/MCP, delegated authorization, confirmation policy or AI-action audit flow exists.
- **Codex-assisted estimate:** 48–80 hours for a safe first version.

3.37 Fireflies transcript processing

Status: Not built.

- No Fireflies API/webhook/client code was found.
- Existing call transcript/intelligence services can be reused after a Fireflies ingestion and identity-matching layer is added.
- **Codex-assisted estimate:** 32–56 hours plus provider access.

3.38 Automatic action creation after calls

Status: Foundation only.

- Call AI can persist summaries/scores and the task API can create assigned work. These parts are not joined into a robust post-call approval workflow.
- Missing: action extraction schema, dedupe/idempotency, human review, assignment rules and source links.
- **Codex-assisted estimate:** 28–48 hours after Fireflies/call source decision.

3.39 CRM updates and follow-up drafting

Status: Partial components.

- WhatsApp AI drafts exist; contacts, activities, calls, tasks and sales-assistant AI are writable components.
- There is no generalized transcript → proposed CRM field changes + follow-up draft → human approval → commit/send workflow.
- **Codex-assisted estimate:** 32–56 hours.

3.40 Permissions, security and GDPR

Status: Strong code foundation; not production-certified.

- JWT sessions, refresh/logout, 2FA, OAuth, role/permission middleware, memberships, location scope, audit logs, consent fields, compliance documents/data-access requests, Helmet, CORS and rate limiting exist.
- Production startup rejects unsafe core configuration. Webhook signatures exist for key inbound providers.
- Risks: legacy role aliases can broaden equivalence; token encryption derives from JWT secret; full tenant-negative test coverage is not complete; no penetration test, DPIA, retention/erasure proof, managed backup evidence or incident runbook was supplied.
- **Codex-assisted estimate:** 72–120 engineering/QA hours plus legal/operations work.
- **Evidence:** authentication/authorization/security/compliance modules, `backend/src/config/index.ts`, tenant-isolation documentation/tests.

3.41 Mobile usability

Status: Responsive implementation present; release evidence missing.

- Pages use responsive layouts/breakpoints and mobile navigation patterns.
- Repository QA notes explicitly identify real desktop/mobile staging verification as pending. No Playwright/device run was completed in this audit.
- **Codex-assisted estimate:** 24–48 QA/fix hours.

3.42 Admin settings

Status: Built / broad v1.

- Team, roles/permissions, packages, integrations, API keys, security, profile/settings, locations, compliance and background-job administration have backend foundations; the active navigation exposes the principal internal settings.
- Some inherited clinic settings remain in the codebase and should be hidden or removed from Mission Control roles.
- **Codex-assisted estimate:** 24–40 hours for cleanup, access review and operations acceptance.

4. Architecture and system connections

Current technical architecture

```
Internal staff browser
-> Next.js 16 + React 19 + TypeScript frontend
-> JWT-authenticated REST API
-> Express 5 + Node.js + TypeScript backend
-> MySQL transactional database
-> optional external services:
    Google APIs / Meta / Twilio / Brevo / Stripe / OpenAI / Google Drive
-> in-process scheduled background-job runner
```

- **Frontend:** Next.js App Router, React, TypeScript, Tailwind CSS; documented as static-export capable.
- **Backend:** Express, TypeScript ESM, MySQL through `mysql2/promise`.

- **Hosting:** documentation names separate staging and production targets, but the actual hosting provider/deploy was not verified.
- **Database:** Mission Control is documented to use its own database (`growth_group_internal_crm` , staging and production variants). It does not currently contain a separate central warehouse.
- **Clinic data flow:** not implemented as a reliable Clinic OS fleet integration. The target flow still needs a versioned API/webhook contract and identity mapping.
- **Isolation:** most service queries use authenticated `clinic_id` ; memberships and switching exist. Good foundation, but full cross-module negative testing is incomplete.
- **APIs/webhooks:** REST modules are extensive. Public forms, website leads, Twilio, WhatsApp and email webhooks exist. API-key authentication supports scoped ingestion.
- **Scheduled sync:** an in-process scheduler persists job state/runs and currently schedules SLA checks, daily rollup, recurring tasks and communication sequences. Marketing connector sync is callable, but no connector sync schedule appears in the job definitions.
- **Errors/failed syncs:** central Express error handling, structured logger, connector failure fields, raw payloads and background job run errors exist. There is no general dead-letter/replay console.
- **Duplicates:** contact duplicate candidates/import handling exist; cross-system duplicate identity management does not.
- **Logging/monitoring:** Winston/request/job logging exists. No deployed APM, centralized log search, uptime alerting or on-call integration was evidenced.
- **Backups/recovery:** backup and restore scripts and pre-deployment guidance exist. Automated encrypted off-site backups and a completed restore drill were not evidenced.
- **Environments:** separate dev/staging/production templates and target domains are documented. Actual separation was not tested.
- **Version control/deployment:** Git/GitHub is in use. No CI/CD workflow, container definition or deployment automation was found in the repository.
- **Takeover readiness:** code and module documentation are substantial, but legacy naming, stale historical docs, provider setup and undocumented hosting state mean another developer would still need verbal/credential handover.

Recommended Clinic OS-to-Mission Control target

Do not share application tables or point Mission Control directly at clinic databases. Use a separate ingestion boundary:

```
Clinic OS tenant
-> signed versioned events and incremental summary APIs
-> Mission Control ingestion service
-> tenant/client identity registry
-> idempotency, validation, retry and reconciliation
-> central performance fact/snapshot tables
-> cross-client dashboards and alerts
```

Minimum controls: scoped credentials per clinic, event IDs, schema versions, cursor/checkpoints, replay, dead-letter state, duplicate detection, data lineage, freshness indicators, per-client sync health and erasure/retention propagation.

5. Data model and real/test data

The schema is broad and real, not a set of frontend-only mocks. Important Mission Control records include:

- accounts/workspaces: `clinic` , `client_account_profile` , `client_account_service` ;
- people/sales: `contact` , `activity` , `pipeline` , `pipeline_stage` , `deal` , `pipeline_deal_movement` , `proposal` ;
- work: `task` , task comments/mentions/attachments, SOPs and strategy logs;
- communication: calls, emails, SMS, WhatsApp conversations/messages/replies;
- platform/operations: users, memberships, roles, permissions, audit logs, API keys, integrations, raw payloads and background-job state/runs;
- intelligence/reporting: growth scores, insights, reports, action plans, campaigns, attribution and manual metrics.

`db.sql` also includes significant sample records. This is appropriate for development, but production creation needs a seed audit. The frontend retains some mock utility files, though core active Mission Control pages examined here use

authenticated APIs. “API-backed” does not prove a live production data feed.

6. Testing and verification

Checks run during this audit

- Backend `npm run build`: **passed**.
- Frontend `npm run typecheck`: **passed**.
- Frontend `npm test`: **passed — 4 files, 13 tests**.
- Frontend lint and production build were started concurrently but did not terminate after several minutes and were stopped. They are **inconclusive**, not passes or confirmed failures.
- `git diff --check` on the report change: passed.

Test coverage reality

The backend contains more than 50 TypeScript integration/smoke test programs covering auth, contacts, pipeline, tasks, client accounts, calls, WhatsApp, Google OAuth/Drive, integrations, reports and tenant behavior. However, the default backend `npm test` script only compiles TypeScript; it does not execute the whole suite. Many backend tests require a database and were not run in this audit.

Frontend automated coverage is small relative to roughly 90 page routes. Browser QA documents exist, but live desktop/mobile/staging evidence remains incomplete. This is a production risk.

7. Completed milestones, current work and blockers

Completed/credible milestones

- isolated Internal CRM repository and documented environment separation;
- fresh Mission Control database model and entity mapping;
- auth/session/2FA/OAuth foundations and production config guard;
- roles, permissions, memberships, team and invitations;
- contacts/leads, imports, duplicate candidates, priority and next actions;
- pipeline/Kanban/deals and movement tracking;
- client account profiles, contacts, packages, services, Drive and account summaries;
- delivery tasks, recurrence, QA, task workspaces, comments, mentions and attachments;
- inbound email webhook, unified inbox and WhatsApp AI/provider foundations;
- calls, transcripts/intelligence routes and sales demo/call tracking fields;
- Google marketing connector implementation for Ads, GA4, GBP and SEO/Search Console;
- audit/compliance/API-key/background-job foundations.

Current work in progress or awaiting review

Recent commits show active work on inbound email, pipeline/task workflow, collaborative task workspaces, sales call/demo tracking, next-best actions, lead priority, audit workflows, growth scores, consent, attribution and marketing connectors. The repository's June Trello notes are stale relative to several of these commits. Without live Trello access, exact board state cannot be certified.

Blocked by another developer

No code evidence reliably identifies work blocked by a particular developer. Repository notes historically assign some Trello cards to `HM`, but this is not enough to map responsibility to a named person.

Blocked by boss/product decisions

- the exact 10-day MVP and launch audience;
- whether Mission Control tasks replace or synchronize with ClickUp;
- accounting/invoice source of truth;
- proposal/SOW/e-sign vendor and workflow;
- finance definitions for revenue, MRR, recognized revenue, costs and profitability;
- the cross-system Clinic OS data contract and data ownership;
- client-health/staff-performance scoring rules;
- alert/briefing delivery channels;
- retention, call recording/transcript and GDPR policies;
- whether legacy clinic-facing modules should be removed, hidden or maintained.

Blocked by access/API/third parties

- live Mission Control hosting, databases and test accounts;
- Trello and design systems;
- Google Ads developer token and OAuth apps;
- Meta app/ad-account permission and review;
- Twilio/WhatsApp numbers/templates;
- Brevo inbound domain/webhook;
- Stripe and accounting system;
- OpenAI production project/budget;
- Google Drive service account/OAuth;
- Fireflies and ClickUp accounts;
- Clinic OS endpoints/credentials and tenant test data.

Discussed but not formally scoped / not built

ClickUp, Fireflies, voice/ChatGPT, support tickets, full proposals/SOW, contract lifecycle, finance/profitability, suppliers/freelancers, executive briefing and the Clinic OS central data layer.

Duplicated/outdated/abandoned areas

- Mission Control is a fork and retains many clinic/patient/treatment/appointment/reputation/billing pages and tables outside the active internal navigation.
- `clinic` means internal account/workspace while many comments/routes still say clinic. Role aliases also map internal and legacy clinical roles.
- Historical Phase 1 documents describe earlier commits and should not be treated as current status.
- Some proposal navigation is inconsistent; one link targets a non-existent `/app/proposals` while current behavior is mostly a filtered pipeline view.

Areas that may need rebuilding

- the central Clinic OS ingestion/warehouse layer needs purposeful architecture, not incremental reuse;
- finance/profitability needs a proper commercial model or accounting integration;
- general credential storage must use a vault/KMS design;
- legacy internal account/tenant naming should eventually be migrated;
- proposal/contract/support capabilities need first-class workflows rather than extra fields on existing records.

8. Launch risks

Five biggest launch blockers

1. No production/staging release evidence with real users, data and backups.
2. No Clinic OS-to-Mission Control feed or central performance data model.
3. Finance, invoices, profitability and supplier/freelancer costs are not complete.
4. Proposal/SOW/contract and post-call automation workflows are missing.
5. Automated/browser/security/tenant/restore test coverage is not sufficient for the size of the application.

Most serious technical risks

- cross-tenant leakage through legacy `clinic_id` assumptions, exports, files or cross-client reporting;
- incorrect business decisions from sample/manual/stale metrics;
- silent provider/scheduled-job failures without deployed observability;
- sensitive transcripts/contact data without proven retention/deletion/backup controls;
- JWT-secret-derived connector encryption complicating safe key rotation;
- accumulated legacy clinic code increasing change and access-control risk.

Most serious delivery risks

- the requested product is much larger than a ten-day completion project;
- UI volume can be mistaken for live/production functionality;
- external access and product decisions can consume more elapsed time than coding;
- ownership is not reliably recorded;
- stale Trello/status documentation may create conflicting priorities.

9. MVP and staged delivery plan

Recommended Mission Control MVP

Include:

- secure internal authentication, RBAC and audit;
- contacts/leads, imports and duplicate review;
- sales pipeline and follow-ups;
- client accounts, contacts, packages/services, health/churn/renewal fields;
- delivery work and task workspaces;
- unified communication history at the current practical level;
- controlled Brevo/WhatsApp/call integrations that pass staging tests;
- Google Drive client folders;
- manual executive/finance summary;
- backup/restore, monitoring and release runbooks.

Defer to later phases:

- full finance/profitability and supplier cost accounting;
- proposal/SOW/e-sign and full contracts;
- ClickUp synchronization;
- Fireflies and automatic post-call actions;
- voice/ChatGPT actions;
- client support portal/tickets;
- full Clinic OS fleet warehouse and cross-client performance reporting;

- sophisticated staff/client scoring.

Next 7 days

1. Deploy/confirm a staging environment from the audited commit.
2. Reconcile live Trello with code and assign every MVP item to a named owner.
3. Create a fresh staging DB and run the backend integration test groups against it.
4. Complete desktop/mobile browser smoke for login, leads, pipeline, client conversion, services, tasks, Drive and inbox.
5. Run tenant-negative tests with two workspaces and multiple roles.
6. Rehearse backup and restore; configure centralized error/uptime alerting.
7. Fix blockers only; freeze new broad feature work.

Next 14 days

- launch a small internal alpha for leads, pipeline, clients and delivery;
- configure and validate Brevo plus one approved messaging/call provider;
- finish renewal/health dashboard behavior and task alerts needed by internal staff;
- remove/hide misleading clinic-facing navigation/routes for internal roles;
- approve the Clinic OS ingestion design and first data contract;
- define finance, proposal and ClickUp source-of-truth decisions.

Next 30 days

- stabilize the internal MVP using staff feedback and usage/error data;
- productionize Google connector schedules and Meta metrics sync if priorities require it;
- build the first narrow Clinic OS account-summary ingestion with reconciliation and freshness status;
- implement either proposal/SOW or finance/accounting integration as the next commercial vertical slice, not both simultaneously;
- scope Fireflies/post-call automation after the core CRM record model is stable.

10. Responsibilities

The repository cannot reliably map “you,” “Michael 2,” other developers or HM . The table below is a recommended allocation to be confirmed by management.

Person/role	Required work
Lead developer / current backend owner	Release gate, database/integration tests, tenant security, production configuration, cross-system architecture and backend fixes.
Michael 2 / frontend owner	Browser/mobile QA, workflow/UI fixes, accessibility, error/empty states and frontend/API contract acceptance.
Other integration/data developer	Clinic OS ingestion, provider sync schedules, reconciliation, warehouse facts and observability.
Usman / marketing team	Supply provider access, account/property/location IDs, campaign/source definitions, KPI/attribution rules and validate client-result accuracy.
Boss/product owner	Approve MVP, priorities, named owners, finance/proposal/ClickUp decisions, data ownership, scoring rules and acceptable manual fallbacks.
Operations/data-protection owner	Backups, incident response, DPIA/ROPA/DPA, retention, DSAR, call/transcript consent and processor reviews.

11. Relevant links

Available from the repository

- Repository: <https://github.com/hailolaa/Internal-CRM>
- Audited commit: <https://github.com/hailolaa/Internal-CRM/commit/f11771b>
- Documented staging frontend: <https://mission-control-staging.thegrowthgroup.com>
- Documented staging API: <https://api-mission-control-staging.thegrowthgroup.com/api>
- Documented production frontend: <https://mission-control.thegrowthgroup.com>
- Documented production API: <https://api-mission-control.thegrowthgroup.com/api>
- Example Trello card — Contacts API: <https://trello.com/c/2vgW0UkM/7-contacts-api-foundation>
- Example Trello card — Pipeline Deals: <https://trello.com/c/4iNtgYAH/18-pipeline-deals-api>

The environment URLs are documented targets and were **not verified live** during this audit.

Missing from the available evidence

- full live Trello board URL/export;
- Figma/design links;
- hosting/deployment project and monitoring links;
- staging and production release identifiers;
- test account details (these should be shared through a password manager, not committed or embedded in this report);
- provider-console links/approval state;
- Clinic OS API/data-contract documentation.

12. Code evidence index

Evidence area	Primary files
Active Mission Control navigation	<code>frontend/lib/navigation.ts</code>
Internal dashboard	<code>frontend/app/app/page.tsx</code> , <code>dashboard/next-action</code> libraries
API module registration	<code>backend/src/app.ts</code>
Production config/security guard	<code>backend/src/config/index.ts</code> , <code>backend/src/index.ts</code>
Leads/contacts	<code>backend/src/modules/contacts/**</code> , <code>contact/lead</code> frontend pages
Pipeline	<code>backend/src/modules/pipeline/**</code> , <code>frontend/app/app/crm/pipeline/page.tsx</code>
Clients/services/Drive	<code>backend/src/modules/client-accounts/**</code> , <code>frontend/app/app/ops/client-accounts/**</code>
Tasks/delivery	<code>backend/src/modules/tasks/**</code> , <code>task/delivery</code> frontend pages
Email/WhatsApp	<code>backend/src/modules/comms/**</code> , <code>backend/src/services/email.service.ts</code>
Calls/AI/transcription	<code>backend/src/modules/calls/**</code> , <code>backend/src/services/openai-call-*.service.ts</code>
Marketing connectors	<code>backend/src/modules/integrations/**</code>
Background automation	<code>backend/src/modules/background-jobs/**</code>
Roles/security/compliance	<code>auth</code> , <code>roles</code> , <code>security</code> , <code>audit-log</code> and <code>compliance</code> modules
Database structures/test seed	<code>backend/db.sql</code> , <code>backend/scripts/migrations/**</code>
Environment/deployment policy	<code>docs/mission-control-environments.md</code> , <code>.env.*.example</code> files

Evidence area	Primary files
Data model/isolation	docs/mission-control-data-model.md , docs/mission-control-data-isolation.md
Automated evidence	backend/src/test/** , frontend *.test.ts , QA docs

13. Final assessment

Mission Control is a substantial working Internal CRM foundation, not a mock-up. Leads, contacts, pipeline, client accounts, services and delivery tasks are the strongest areas and are suitable for a controlled internal alpha once the release gate is completed.

The full vision in the boss's message is not close to production-complete because finance/profitability, proposals/contracts, support, Fireflies/ClickUp, central Clinic OS data ingestion and production operational proof remain incomplete. The most honest current planning figure is **52% production-ready against the full requested scope**, with **420–650 development hours plus 150–230 testing/hardening hours remaining when Codex is used continuously alongside the developers**. Provider approval, access and management decision delays are additional elapsed time.

The best next move is not to open more feature fronts. It is to launch the core internal workflow safely, obtain real staff usage, and in parallel approve the cross-system data architecture and one next commercial vertical slice.